

Análise Comparativa de Arquiteturas para Classificação de Textos

Carlos da Mota Bergueira¹, Diego Jefferson Mendes Silva¹, Filipa Araújo Pereira¹, and Rui Manuel Martins Marques Rodrigues¹

Universidade do Minho, Braga, Portugal
pg11605@alunos.uminho.pt, pg59999@alunos.uminho.pt,
pg42201@alunos.uminho.pt, pg7942@alunos.uminho.pt

Abstract. Este trabalho aborda o problema de classificação de origem textual em cenário multiclasse, distinguindo textos escritos por humanos de textos gerados automaticamente e, neste último caso, identificando a família do modelo gerador (Anthropic, Google, Meta ou OpenAI). Para esse fim, foi definida uma pipeline supervisionada com recolha e integração de múltiplos datasets públicos, balanceamento de classes e divisão estratificada de treino e avaliação. No pré-processamento, foram exploradas representações vetoriais baseadas em TF-IDF (palavra e carácter), complementadas por *features* estilométricas e estatísticas.

Do ponto de vista metodológico, foram comparadas diferentes famílias de modelos: um *baseline* linear regularizado, uma rede neuronal implementada de raiz em NumPy, uma rede neuronal em PyTorch, um Transformer encoder treinado para a tarefa e um modelo pré-treinado RoBERTa ajustado por *fine-tuning*. A comparação foi conduzida com métricas de classificação multiclasse, com ênfase em *accuracy* e *F1*-macro, analisando também estabilidade por classe.

Como principal contribuição, o estudo fornece uma análise comparativa entre arquiteturas de diferentes níveis de complexidade, discutindo o compromisso entre custo computacional, capacidade representacional e robustez na classificação de origem textual.

Keywords: Deep Learning · Text Classification · Neural Networks · Transformers · Natural Language Processing · Machine Learning · Artificial Intelligence

Código e reprodutibilidade. O código-fonte, instruções de execução e artefatos experimentais estão disponíveis em: <https://github.com/MIA-CDFR/AP>.

1 Introdução

A classificação de textos é uma tarefa fundamental em Processamento de Linguagem Natural (PLN), com aplicações que vão desde a filtragem de spam até a análise de sentimentos e detecção de autoria. Com o advento de modelos de linguagem cada vez mais sofisticados, a distinção entre textos gerados por humanos e por máquinas tornou-se um desafio crescente.

Este trabalho tem como objetivo desenvolver e comparar modelos de aprendizagem supervisionada para classificar textos segundo a sua origem, distinguindo produção humana de produção automática. Para textos de origem automática, o trabalho visa identificar a família do modelo gerador, considerando especificamente as famílias de modelos Anthropic, Google, Meta e OpenAI.

Para avaliar diferentes estratégias de representação textual e arquiteturas de classificação, foram desenvolvidos cinco modelos distintos: um *baseline* linear para comparação, dois modelos DNN (um com implementação NumPy [1] pura e outro com PyTorch [2] para validação), e dois modelos baseados em *Transformers* (um genérico treinado do zero e outro baseado em *fine-tuning* de BERT pré-treinado). Esta abordagem permite análise comparativa entre métodos tradicionais e modernos de PLN.

- **Baseline Linear Regularizado:** Implementação de um modelo linear de referência (Regressão Logística com regularização L2) utilizando apenas NumPy, para estabelecer um *benchmark* de desempenho e comparação com métodos mais complexos.
- **Deep Neural Network (DNN) com NumPy:** Implementação de uma rede neural profunda multiclasse com múltiplas camadas ocultas e funções de ativação não lineares, desenvolvida do zero utilizando apenas NumPy. O modelo inclui mecanismos de controlo de overfitting (*dropout*, *early stopping*) e técnicas de otimização gradiente descendente estocástico (SGD).
- **Deep Neural Network (DNN) com PyTorch:** Versão re-implementada da DNN anterior utilizando PyTorch, para validação dos resultados e aproveitamento de GPU. Mantém arquitetura equivalente ao modelo NumPy para comparação justa de performance.
- **Transformer Genérico:** Implementação de um modelo *Transformer* para classificação de texto, utiliza arquitetura baseada em auto-atenção multi-cabeça e camadas *feedforward*, com opção de treino do zero ou inicialização com pesos pré-treinados.
- **BERT Fine-tuned:** Utilização do modelo BERT pré-treinado (**roberta-base** [3]) para classificação, realizando *fine-tuning* específico para a tarefa de classificação de origem de textos, aproveitando conhecimento de linguagem pré-existente.

2 Seleção de Datasets e Preprocessamento

2.1 Fontes de Dados

Para a construção dos conjuntos de treino e teste, foram selecionados datasets públicos do Hugging Face que cobrem diferentes origens textuais: produção humana e produção automática por diferentes famílias de modelos de linguagem. A seleção teve como objetivo garantir representatividade por classe e diversidade de estilos textuais.

Os dados utilizados provêm das seguintes fontes:

- *MLNTeam-Unical/OpenTuringBench* [4]: fornece textos gerados pelos modelos da família Meta e Google, com 37.283 amostras por classe, totalizando 74.566 amostras. O dataset completo contém 5 classes adicionais (Intel, Microsoft, Mistral AI, Qwen, Upstage) com 186.415 amostras, que não foram utilizadas.
- *Dahoas/instruct-synthetic-prompt-responses* [5]: fornece textos gerados por modelos da família OpenAI, com 33.203 amostras.
- *artem9k/ai-text-detection-pile* [6]: fornece textos de origem humana, com 1.028.146 amostras.
- *Anthropic/persuasion* [7]: fornece textos gerados por modelos da família Anthropic, com 3.360 amostras.

Dados adicionais foram fornecidos durante o desenvolvimento experimental:

- *Dataset de Exemplos*: disponibilizado pelo professor na fase inicial, contendo 125 amostras com distribuição: Anthropic (23), Google (16), Meta (17), OpenAI (17) e Human (52). Utilizado na 1ª submissão como parte do treino.
- *Dataset Revelado 1ª Submissão*: dataset com labels revelados após a 1ª submissão, contendo 100 amostras com distribuição: Anthropic (17), Google (17), Human (34), Meta (18) e OpenAI (14). Integrado ao treino na 2ª submissão.
- *Dataset Revelado 2ª Submissão*: dataset com labels revelados após a 2ª submissão, contendo 100 amostras com distribuição: Anthropic (16), Google (18), Human (34), Meta (16) e OpenAI (16). Integrado ao treino na 2ª submissão.

Face ao acentuado desequilíbrio entre classes no conjunto de dados bruto (1.028.146 amostras da classe *Human* versus 3.360 da classe *Anthropic*), foi adotada uma estratégia de balanceamento no conjunto de treino de cada submissão, com o objetivo de mitigar o viés em favor da classe majoritária. Esse procedimento combinou *undersampling* da classe *Human* com *oversampling* das classes minoritárias. Adicionalmente, o número de amostras por classe foi limitado a 5.000, resultando em um conjunto balanceado com 5.000 instâncias por classe (25.000 amostras no total, em cada versão do conjunto de treino). Como pode ser observado na Fig. 1.

Para a divisão entre treino e teste, foi utilizada a estratégia de divisão 80/20, onde 80% dos dados foram utilizados para treino e 20% para teste. A divisão foi realizada de forma estratificada, garantindo que a proporção de amostras por classe se mantivesse uniforme em ambos os conjuntos.

2.2 Dataset de Validação

O dataset de validação variou conforme as diferentes submissões:

- *1ª Submissão*: 125 amostras do dataset de exemplos fornecido pelo professor, distribuídas como: Anthropic (23), Google (16), Meta (17), OpenAI (17) e Human (52).

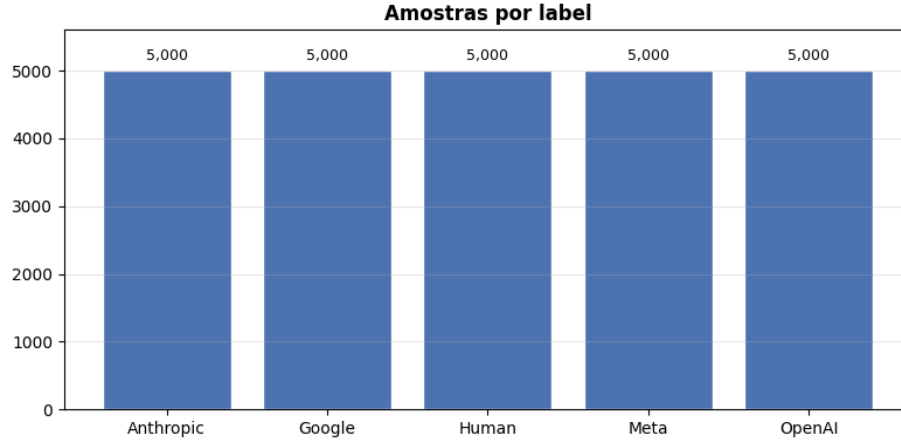


Fig. 1. Distribuição das classes no conjunto de treino e teste.

- *2ª Submissão*: 100 amostras do dataset revelado da 1ª submissão, distribuídas como: Anthropic (17), Google (17), Human (34), Meta (18) e OpenAI (14).
- *3ª Submissão*: 100 amostras do dataset revelado da 2ª submissão, distribuídas como: Anthropic (16), Google (18), Human (34), Meta (16) e OpenAI (16).

2.3 Preprocessamento

O preprocessamento de texto foi adaptado conforme o tipo de modelo utilizado, considerando as diferentes estratégias de representação textual e os requisitos computacionais de cada arquitetura.

NumPy DNN O modelo *Deep Neural Network (DNN)* implementado com NumPy utiliza uma abordagem baseada em representações vetoriais híbridas, combinando *term frequency-inverse document frequency (TF-IDF)* com *features* manuais. O preprocessamento inclui duas etapas:

1. *Representação TF-IDF*: O texto é convertido em vetores TF-IDF em dois níveis de análise. A análise ao nível de palavras utiliza normalização L2 com n-gramas unigrama e bigrama, enquanto a análise ao nível de caracteres (1-gramas a 3-gramas) captura padrões morfológicos. Ambas as representações são normalizadas pela norma L2 para garantir comparabilidade entre amostras de comprimento variável.
2. *Features Manuais*: Complementarmente, são extraídas *features* estatísticas e estilométricas do texto original: comprimento em caracteres, número de palavras, número de sentenças, comprimento médio de palavras, comprimento médio de sentenças, diversidade lexical (*type-token ratio*), proporção

de palavras raras (*hapax ratio*), frequência de marcas de pontuação e estatísticas de repetição de palavras. Estas features são normalizadas (*z-score*) antes da concatenação com a representação TF-IDF.

PyTorch DNN O modelo *Deep Neural Network (DNN)* implementado em PyTorch utiliza entrada vetorial baseada em TF-IDF, mantendo consistência com a formulação de classificação multiclasse por vetores densos de texto. O préprocessamento segue o seguinte pipeline:

1. *Normalização textual*: os textos são normalizados para reduzir variabilidade superficial (e.g., caixa e ruído), mantendo conteúdo lexical relevante para classificação.
2. *Vetorização TF-IDF*: os textos de treino são transformados com *TfidfVectorizer*, configurado com *max_features=12000*, *ngram_range=(1,2)*, *min_df=5* e remoção de *stop words* em inglês. O mesmo vetorizador ajustado no treino é aplicado ao conjunto de avaliação.
3. *Conversão para tensores*: a matriz esparsa TF-IDF é convertida para representação densa por amostra e utilizada como entrada da DNN em PyTorch.

Transformer (Tokenizador Genérico) O modelo Transformer utiliza um tokenizador genérico para vocabulário compartilhado. O processamento inclui:

1. *Tokenização com AutoTokenizer*: Utiliza-se um tokenizador pré-configurável (AutoTokenizer da biblioteca Transformers) que normaliza texto, aplica case-folding e segmenta em sub-palavras ou tokens segundo o esquema do tokenizador selecionado.
2. *Codificação de Sequências*: As sequências de tokens são codificadas em índices do vocabulário, com comprimento máximo fixo (e.g., 128 tokens). Sequências mais curtas são preenchidas com tokens especiais de padding [PAD], e sequências mais longas são truncadas.
3. *Tensores de Atenção*: Com base no tokenizador, são construídas máscaras de atenção para indicar posições de padding ao modelo, permitindo que o Transformer ignore tokens de padding durante o cálculo de auto-atenção.

BERT (Fine-tuning) O modelo BERT utiliza um tokenizador e configuração pré-treinados do modelo base. O préprocessamento específico para BERT é o seguinte:

1. *Tokenização BERT*: Utiliza o tokenizador específico do modelo pré-treinado **roberta-base** [3], que normaliza a entrada e aplica tokenização BPE. Tokens especiais do RoBERTa [3], como <s>, </s> e <pad>, são inseridos automaticamente conforme o formato esperado pelo modelo.
2. *Sequências de Entrada*: O texto é convertido em sequências de tokens no formato de entrada do RoBERTa [3], com tokens especiais no início e no fim da sequência. As sequências são truncadas ou preenchidas até um comprimento máximo (*default* 512 tokens).

3. *Embeddings Posicionais*: O modelo adiciona embeddings posicionais aos *embeddings* de *token* antes do processamento pelas camadas *Transformer*. No caso do RoBERTa [3], não são utilizados *token type embeddings* (*embeddings* de segmento) como no BERT clássico.

2.4 Formulação Matemática

Nesta subseção, sintetizam-se as principais formulações utilizadas no pré-processamento e no treino dos modelos.

TF-IDF e normalização Para um termo t num documento d , com coleção $\mathcal{D}$:

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}, \quad \text{idf}(t) = \log \left(\frac{1 + |\mathcal{D}|}{1 + \text{df}(t)} \right) + 1,$$

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t).$$

Após vetorização, aplica-se normalização L2 por amostra:

$$\mathbf{x} \leftarrow \frac{\mathbf{x}}{\|\mathbf{x}\|_2}.$$

Para *features* manuais, usa-se padronização *z-score* no treino:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j + \varepsilon}.$$

DNNs (NumPy e PyTorch) Uma camada densa é dada por:

$$\mathbf{h}^{(l)} = \phi(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}),$$

com $\phi(\cdot) = \max(0, \cdot)$ (ReLU). Na saída multiclasse:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^C \exp(z_j)}.$$

A função de perda usada para classificação multiclasse é:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^C y_k \log p_k.$$

Transformer e RoBERTa No mecanismo de autoatenção:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V}.$$

No *Transformer* genérico, após o encoder, usa-se *mean pooling* mascarado:

$$\mathbf{h}_{\text{pool}} = \frac{\sum_{i=1}^T m_i \mathbf{h}_i}{\sum_{i=1}^T m_i},$$

onde $m_i \in \{0, 1\}$ indica tokens válidos (não *padding*).

No RoBERTa *fine-tuning* [3], a cabeça de classificação aplica projeção linear sobre a representação contextual e otimiza novamente a perda de entropia cruzada multiclasse.

3 Arquiteturas de Modelos

Esta seção descreve as arquiteturas efetivamente utilizadas nos quatro modelos principais do trabalho, com ênfase nas camadas, dimensões internas e estratégia de classificação multiclasse.

3.1 NumPy DNN

O modelo *Deep Neural Network* implementado de raiz em NumPy é uma rede *feedforward* totalmente ligada com três camadas ocultas, seguindo a topologia: $\text{input} \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow C$, onde C representa o número de classes. Entre camadas densas são aplicadas ativações ReLU e regularização por *dropout* com taxas 0.2, 0.1 e 0.1. A camada de saída usa *Softmax* para obter distribuições de probabilidade por classe. Os pesos das camadas densas são inicializados com estratégia tipo He e incluem penalização L2.

3.2 PyTorch DNN

O modelo DNN em PyTorch preserva a mesma ideia arquitetural da versão NumPy, mas com implementação em `nn.Sequential`. A arquitetura é definida por: $\text{Linear}(d, 512) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow \text{Linear}(512, 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(256, 128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.2) \rightarrow \text{Linear}(128, C)$. Esta configuração permite comparar diretamente o efeito da framework de treino, mantendo profundidade e capacidade semelhantes à versão NumPy.

3.3 Transformer Genérico

O classificador Transformer é *encoder-only*, composto por embedding de tokens, embedding posicional aprendível e uma pilha de blocos *Transformer Encoder*. A configuração utilizada foi: comprimento máximo de sequência $T = 512$, dimensão do modelo $d_{\text{model}} = 512$, 16 cabeças de atenção, 6 camadas encoder, *dropout* 0.2 e ativação GELU no *feedforward*. A representação final da sequência é obtida por *mean pooling* mascarado (ignorando *padding*), seguida de *dropout* e camada linear para logits multiclasse.

3.4 BERT Fine-tuning (RoBERTa-base)

O quarto modelo utiliza *fine-tuning* de um encoder pré-treinado, concretamente **roberta-base** [3], através de `AutoModelForSequenceClassification`. A arquitetura base possui 12 camadas Transformer, dimensão oculta 768 e 12 cabeças de atenção. Sobre o encoder pré-treinado é adicionada uma cabeça de classificação para C classes. O treino ajusta fim-a-fim tanto o encoder como a cabeça final, permitindo transferir conhecimento linguístico pré-treinado para a tarefa de classificação de origem textual.

Table 1. Comparação de desempenho dos modelos no conjunto de avaliação.

Modelo	Accuracy	Macro-F1	Macro-Precision	Macro-Recall
Baseline Linear	—	—	—	—
NumPy DNN	0.3700	0.3171	0.3885	0.3706
PyTorch DNN	0.3300	0.2854	0.2935	0.3092
Transformer Genérico	0.5900	0.5097	0.6712	0.5456
RoBERTa-base (fine-tuning)	0.7900	0.7487	0.7892	0.7791

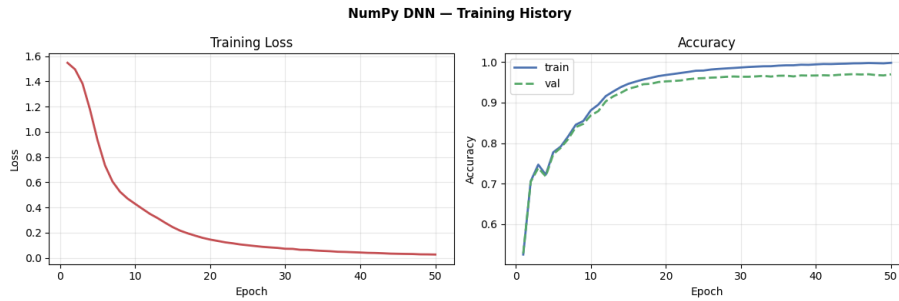
4 Treino e Avaliação

Todos os modelos foram treinados com divisão estratificada 80/20 entre treino e avaliação, preservando a proporção de classes. A avaliação baseou-se em métricas de classificação multiclasse, com ênfase em *accuracy* e, nos modelos baseados em *Trainer* (Transformer e RoBERTa), em *F1-macro* para seleção do melhor *checkpoint*.

4.1 NumPy DNN

O modelo NumPy DNN foi treinado com mini-batches (`batch_size = 32`), até 100 épocas, taxa de aprendizagem inicial de 0.005 e *early stopping* com paciência 5. A função de custo utilizada foi *Categorical Cross-Entropy*, com possibilidade de ponderação por classe para mitigar desequilíbrio residual. O processo de treino monitoriza perda média por época, *accuracy* de treino e *accuracy* de validação, além de acurácia por classe no conjunto de avaliação.

No fim do treino, os resultados são visualizados na Fig. 2.

**Fig. 2.** Resultados do treino do modelo NumPy DNN.

4.2 PyTorch DNN

O modelo PyTorch DNN foi treinado com *Cross-Entropy Loss* e otimizador Adam, usando `learning_rate = 10-3`, `batch_size = 32` e até 10 épocas. Foi

aplicado *early stopping* com paciência 3, com monitorização de *accuracy* de validação por época. Para análise complementar, foram calculadas métricas globais no conjunto de avaliação e acurácia por classe.

No fim do treino, os resultados são visualizados na Fig. 3.

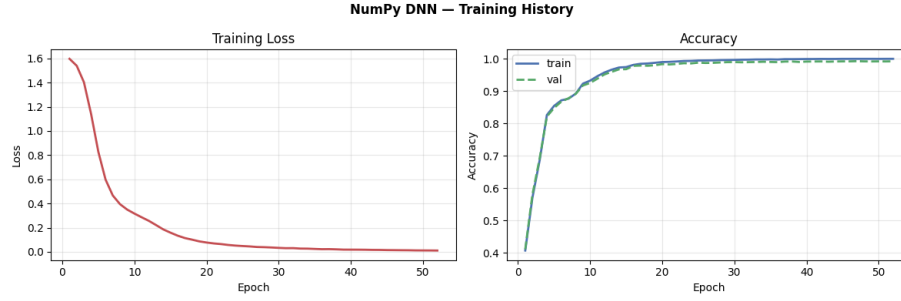


Fig. 3. Resultados do treino do modelo PyTorch DNN.

4.3 Transformer Genérico

O Transformer genérico foi treinado com a API *Trainer* da biblioteca *Transformers*, com os seguintes hiperparâmetros principais: 3 épocas, `batch_size = 32`, `learning_rate =  $2 \times 10^{-4}$` , `weight_decay` de 0.01, `warmup_ratio` de 0.1 e `scheduler` cosseno. A avaliação foi executada ao fim de cada época, com retenção automática do melhor *checkpoint* segundo *F1-macro*.

No fim do treino, os resultados são visualizados na Fig. 4.

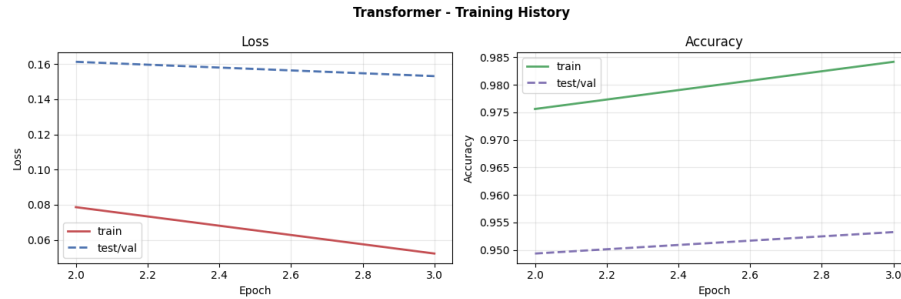


Fig. 4. Resultados do treino do modelo Transformer Genérico.

4.4 BERT Fine-tuning (RoBERTa-base)

O modelo RoBERTa-base [3] foi ajustado por *fine-tuning* com *Trainer*, utilizando 3 épocas, `batch_size` = 32, `learning_rate` = 2×10^{-5} , *weight decay* de 0.01, comprimento máximo de sequência 256 e acumulação de gradientes em 2 passos. A avaliação foi realizada por época, com carregamento automático do melhor modelo no final do treino segundo *F1*-macro.

Em termos comparativos, a avaliação final considerou consistência entre desempenho global (*accuracy*/*F1*-macro) e estabilidade por classe, permitindo analisar o compromisso entre modelos mais leves (DNNs) e modelos baseados em *Transformers*.

5 Discussão

Os resultados obtidos evidenciam diferenças claras de desempenho entre as arquiteturas avaliadas. O melhor resultado foi alcançado pelo modelo RoBERTa-base ajustado por *fine-tuning*, com *accuracy* de 0.7900 e *F1*-macro de 0.7487, além de *macro-precision* de 0.7892 e *macro-recall* de 0.7791. Em seguida, o Transformer genérico apresentou desempenho intermediário, com *accuracy* de 0.5900 e *F1*-macro de 0.5097, confirmando que arquiteturas baseadas em atenção já oferecem ganho substancial em relação às DNNs baseadas em TF-IDF, ainda que permaneçam abaixo do modelo pré-treinado. Esse resultado reforça a vantagem do *transfer learning* quando há disponibilidade limitada de dados rotulados e necessidade de maior capacidade representacional.

Entre os modelos vetoriais, o NumPy DNN superou o PyTorch DNN nesta avaliação, com *accuracy* de 0.3700 contra 0.3300 e *F1*-macro de 0.3171 contra 0.2854. Embora ambas as abordagens apresentem menor custo computacional e maior simplicidade de interpretação, os resultados sugerem que, neste cenário, as representações TF-IDF combinadas com arquiteturas densas não capturaram de forma suficiente os padrões semânticos necessários para discriminar com robustez a origem textual. Adicionalmente, a análise por classe permanece essencial, pois métricas agregadas podem ocultar degradações relevantes em classes específicas; por isso, a leitura conjunta de *accuracy*, *F1*-macro e estabilidade por classe deve orientar a seleção final do modelo.

6 Limitações e Ameaças à Validade

Este estudo apresenta limitações que devem ser consideradas. Primeiro, a avaliação foi baseada numa divisão estratificada única (80/20), o que pode introduzir variância associada à partição escolhida. Segundo, parte dos dados foi incorporada de forma incremental ao longo das submissões, podendo gerar diferenças de distribuição entre fases experimentais. Terceiro, apesar de terem sido comparadas arquiteturas diversas, nem todos os modelos partilham exatamente a mesma representação de entrada, o que pode afetar a comparabilidade direta.

Como ameaças à validade externa, os resultados podem não generalizar integralmente para domínios textuais distintos (e.g., textos jurídicos, biomédicos ou multilíngues). Como ameaça à validade interna, escolhas de hiperparâmetros e critério de paragem antecipada podem influenciar significativamente o desempenho reportado.

7 Conclusão e Trabalho Futuro

Este trabalho apresentou uma comparação sistemática entre abordagens lineares, DNNs e *Transformers* para classificação de origem textual. A análise conjunta dos modelos permite concluir que arquiteturas mais simples permanecem competitivas em custo-benefício, enquanto modelos *Transformer*-based oferecem maior capacidade representacional em cenários de maior complexidade semântica.

Como trabalho futuro, recomenda-se: (i) realizar validação cruzada com múltiplas sementes para reduzir a variância experimental; (ii) investigar calibração de probabilidades e análise de incerteza; (iii) explorar técnicas de *data augmentation* textual e *domain adaptation*; (iv) incluir testes estatísticos de significância para reforçar a robustez das conclusões; (v) avaliar abordagens de *few-shot learning* com LLMs; e (vi) investigar estratégias híbridas que combinem modelos DNN e *Transformers*.

References

1. Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., Fernandez del Rio, J., Wiebe, M., Peterson, P., Gerard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., Oliphant, T.E.: Array programming with NumPy. *Nature* **585**(7825), 357–362 (2020). <https://doi.org/10.1038/s41586-020-2649-2>
2. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: *Advances in Neural Information Processing Systems* 32, pp. 8024–8035. Curran Associates, Inc. (2019). <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
3. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019). <https://doi.org/10.48550/arXiv.1907.11692>
4. MLNTeam-Unical: OpenTuringBench. Hugging Face Datasets. <https://huggingface.co/datasets/MLNTeam-Unical/OpenTuringBench>, último acesso em 2026/03/29.

5. Dahoas: instruct-synthetic-prompt-responses. Hugging Face Datasets. <https://huggingface.co/datasets/Dahoas/instruct-synthetic-prompt-responses>, último acesso em 2026/03/29.
6. artem9k: ai-text-detection-pile. Hugging Face Datasets. <https://huggingface.co/datasets/artem9k/ai-text-detection-pile>, último acesso em 2026/03/29.
7. Anthropic: persuasion. Hugging Face Datasets. <https://huggingface.co/datasets/Anthropic/persuasion>, último acesso em 2026/03/29.